AICE2005 - Systematic Design

Formal Specification and Verification

Asieh Salehi Fathabadi
University of Southampton
2025-2026

Course Introduction

- Lecture 1: Introduction to Formal Methods
- Lecture 2: Formal Specification Techniques
- Lecture 3: Mathematical Foundations
- Lecture 4: Formal Verification Methods
- Lecture 5: Tools and Applications

LECTURE 3:

Mathematical Foundations

Lecture 3 Overview

- Review & Mathematical Foundations (Bank case study)
- Set Theory & Relations
- Logic Foundations
- Temporal Logic

Learning Objectives:

- Apply set theory operations in formal specifications
- Construct logical formulas using first order logic
- Understand mathematical foundations underlying Event-B

Recap: Safety Requirements and Event-B Specification

- **Safety Requirement** identification
- **Safety properties** specification
- **First Event-B model** construction
- **Platform Screen Door Control System**
  - **System Goal:**
    Ensure passenger safety at station platforms.



* The figure is guided AI-generated

```
CONTEXT DoorContext

SETS DOOR_STATE TRAIN_STATE

CONSTANTS OPEN, CLOSED PRESENT, ABSENT, MOVING
AXIOMS
    partition(DOOR_STATE, {OPEN}, {CLOSED})
    partition(TRAIN_STATE, {PRESENT}, {ABSENT}, {MOVING})

END
```

```
MACHINE DoorControl

SEES DoorContext

VARIABLES door train

INVARIANTS
    door ∈ DOOR_STATE
    train ∈ TRAIN_STATE
    door = OPEN ⇒ train = PRESENT    // SR-1, SR-2
    train = MOVING ⇒ door = CLOSED    // SR-5

INITIALISATION
    door := CLOSED
    train := ABSENT
```

```
EVENTS
OpenDoors ≙ when
    train = PRESENT then
    door := OPEN
end

CloseDoors ≙
    any t where
        t ∈ TRAIN_STATE then
        door := CLOSED
end

TrainArrives ≙ when
    train = ABSENT then
    train := PRESENT
end

TrainDeparts ≙ when
    door = CLOSED then
    train := MOVING
end
END
```

```
MACHINE DoorControl_R1 REFINES DoorControl
VARIABLES door train sensor
INVARIANTS
    sensor ∈ BOOL
    sensor = TRUE ⇔ train ∈ {PRESENT}
EVENTS
OpenDoors refines OpenDoors ≙ when
    sensor = TRUE
    train = PRESENT then
    door := OPEN end
CloseDoors refines CloseDoors ≙ any t where
    door = OPEN
    t ∈ TRAIN_STATE then
    door := CLOSED end
TrainArrives refines TrainArrives ≙ when
    train = ABSENT then
    train := PRESENT end
TrainDeparts refines TrainDeparts ≙ when
    door = CLOSED then
    train := MOVING
    sensor := FALSE end
DetectTrain ≙ when  // new event
    sensor = FALSE
    train = PRESENT then
    sensor := TRUE end
END
```

Set Theory Fundamentals

Basic Set Operations:

- Union: $A \cup B = \{x \mid x \in A \vee x \in B\}$

Example: $\{1,2\} \cup \{2,3\} = \{1,2,3\}$

- Intersection: $A \cap B = \{x \mid x \in A \wedge x \in B\}$

Example: $\{1,2\} \cap \{2,3\} = \{2\}$

- Difference: $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$

Example: $\{1,2,3\} \setminus \{2\} = \{1,3\}$

- Power Set: $\mathbb{P}(A) = \{S \mid S \subseteq A\}$

Example: $\mathbb{P}(\{1,2\}) = \{\varnothing, \{1\}, \{2\}, \{1,2\}\}$

Set Theory Fundamentals

Relations and Functions:

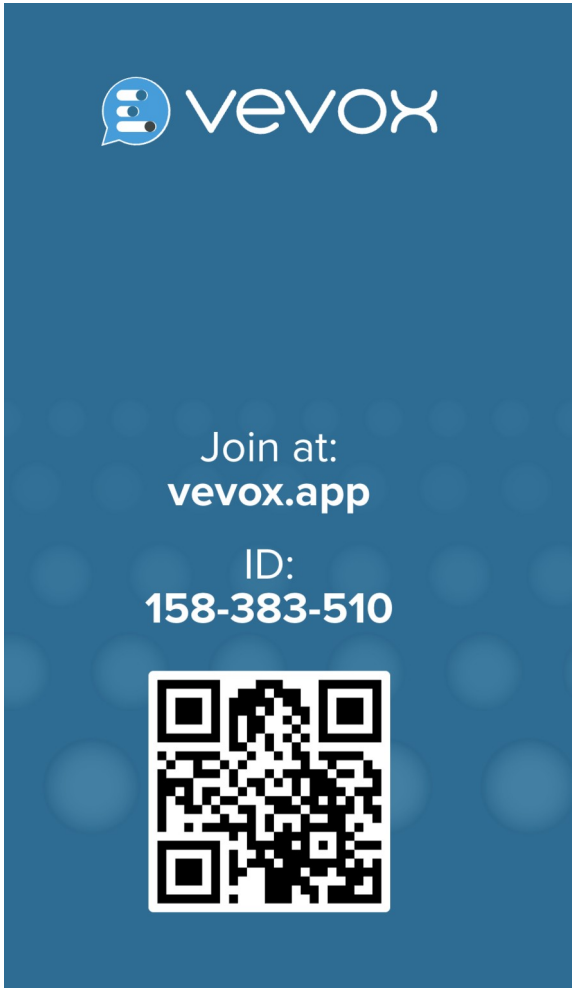
Relation: $R \subseteq A \times B$ (subset of Cartesian product)
Domain: $dom(R) = \{a \mid \exists b \bullet (a,b) \in R\}$
Range: $ran(R) = \{b \mid \exists a \bullet (a,b) \in R\}$
Function: Relation where each input maps to exactly one output

B/Event-B Relation Types:

Relation: $A \leftrightarrow B$ | Any relation from A to B
Partial Function: $A \nrightarrow B$ | Each input maps to at most one output
Total Function: $A \rightarrow B$ | Each input maps to exactly one output
Injection: $A \rightarrowtail B$ | One-to-one mapping
Surjection: $A \twoheadrightarrow B$ | Onto mapping
Bijection: $A \rightarrowtail\!\!\!\!\twoheadrightarrow B$ | One-to-one and onto

Let's think and reflect

Set Theory Exercise



Given

enrolled = { (alice, math), (bob, math), (charlie, chemistry) }

Find

- dom(enrolled) = ?
- ran(enrolled) = ?

Let's think and reflect

vevox

Join at:
vevox.app

ID:
147-644-612

Function Types Exercise

Given

A = { 1, 2, 3 }
B = { x, y, z }

Q1: { 1→x, 2→y }
Q2: { 1→x, 2→x, 3→x }
Q3: { 1→x, 1→y }
Q4: { 1→x, 2→y, 3→z }

Fill with Yes or No

Question	Function	Total	Injective	Surjective	Bijective
Q1	?	?	?	?	?
Q2	?	?	?	?	?
Q3	?	?	?	?	?
Q4	?	?	?	?	?

Relations in Formal Specifications

Set Comprehension in B/Event-B:

-- General form: {expression | predicate}

- $\{x^2 \mid x \in \mathbb{N} \land x < 5\} = \{0, 1, 4, 9, 16\}$

-- Set operations in Event-B context

- $valid_accounts = \{acc \mid acc \in ACCOUNT \land balance(acc) \geq 0\}$

Relational Operations:

-- Relational composition (R ; S)

- parent = { (john, mary), (mary, susan) }
- grandparent = parent ; parent = { (john, susan) }

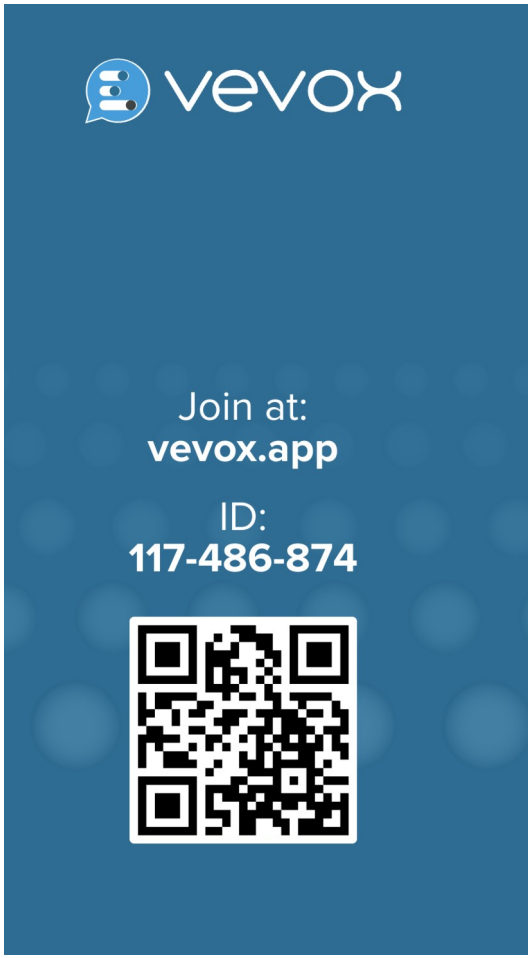
-- Relational inverse (R^{-1})

- teaches = { (prof_smith, math), (prof_jones, physics) }
- taught_by = $teaches^{-1}$ = { (math, prof_smith), (physics, prof_jones) }

-- Domain/Range restriction

- $R \lhd S = \{ (x,y) \mid (x,y) \in R \land x \in S \}$ *-- Domain restriction*
- $R \rhd S = \{ (x,y) \mid (x,y) \in R \land y \in S \}$ *-- Range restriction*

Let's think and reflect



Relations Exercise

Given

- accounts = {acc1, acc2, acc3}
- customers = {alice, bob, charlie}
- owns = {alice ↦ acc1, alice ↦ acc2, bob ↦ acc3}
- balance = {acc1 ↦ 1000, acc2 ↦ 500, acc3 ↦ 2000}

Find

- Customers balance relation ?
- Customers with account balance less than 1500 ?

Logic Foundations

Propositional Logic in Formal Methods:

- Negation: $\neg P$ | not P | True when P is false
- Conjunction: $P \wedge Q$ | P & Q | True when both P and Q are true
- Disjunction: $P \vee Q$ | P or Q | True when P or Q (or both) true
- Implication: $P \rightarrow Q$ | $P \Rightarrow Q$ | False only when P true and Q false
- Equivalence: $P \leftrightarrow Q$ | $P \Leftrightarrow Q$ | True when P and Q same truth value

Predicate Logic in B/Event-B:

- Universal:

  $\forall x \bullet P(x)$ | $\forall x \cdot (x \in S \Rightarrow P(x))$ | For all x in S, P(x)
- Existential:

  $\exists x \bullet P(x)$ | $\exists x \cdot (x \in S \wedge P(x))$ | Exists x in S such that P(x)

Logic Example

-- Banking system invariants

∀acc· (acc ∈ accounts ⇒balance(acc) ≥ overdraft_limit(acc))

∃mgr· (mgr ∈ managers ∧ responsible_for(mgr, accounts))

card ∈ accepted_cards ⇒valid_pin_entered = TRUE

-- Event guards using logic

ANY customer, account WHERE

customer ∈ customers ∧

account ∈ accounts ∧

owns(customer, account) ∧

balance(account) ≥ amount

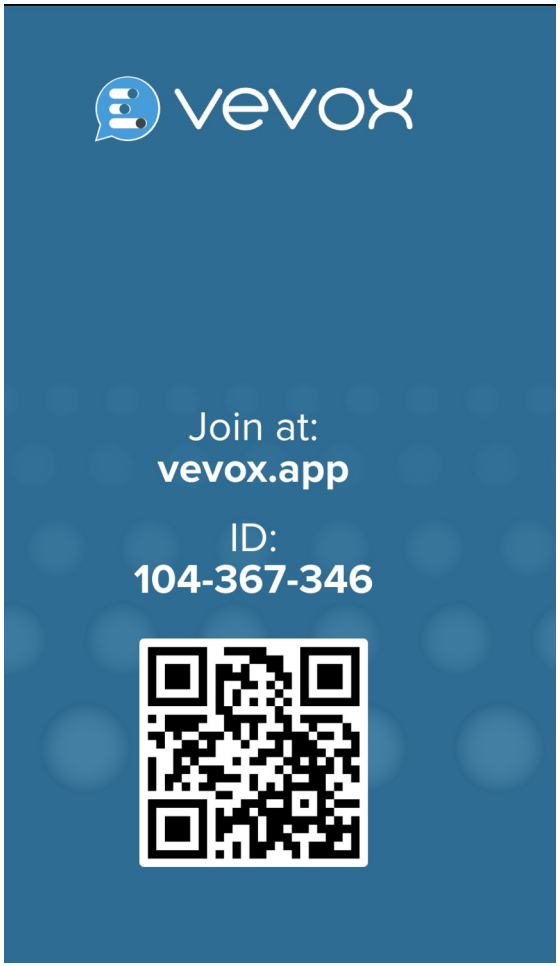
THEN

balance := balance <+ {account ↦ (balance(account) - amount)}

END

Let's think and reflect

Logic Exercise



Context
SETS
  CUSTOMERS
  ACCOUNTS

Machine
VARIABLES
  customers
  accounts
  owns
  balance

INVARIANTS
  inv1: customers ⊆ CUSTOMERS
  inv2: accounts ⊆ ACCOUNTS
  inv3: owns ⊆ customers × accounts
  inv4: balance ∈ accounts → ℤ

Invariant specifying
"Every customer owns at least one account"

?

Invariant specifying
"No account can have negative balance"

?

Linear Temporal Logic (LTL)

Why Temporal Logic?

- Static Logic Limitation: Classical logic describes single states
- Dynamic Systems: Real systems change over time through events
- Temporal Properties: "Eventually," "always," "until," "next"
- Event-B Connection: Events cause state transitions

LTL Operators:

- Next: ○φ | φ holds after next event
  ○(balance > 0)
- Eventually: ◊φ | φ will hold after some events
  ◊(transaction_complete)
- Always: □φ | φ holds after all event sequences
  □(balance ≥ overdraft_limit)
- Until: φ U ψ | φ holds until ψ becomes true
  authenticating U authenticated

LTL Examples

Safety Properties ("Bad things never happen")

- -- *Mutual exclusion in concurrent ATM access*
- □(¬(atm1_processing ∧ atm2_processing ∧ same_account))

- -- *Account balance never goes below limit*
- □(∀acc·(acc ∈ accounts ⇒balance(acc) ≥ overdraft_limit(acc)))

- -- *System security*
- □(sensitive_operation ⇒ authenticated)

Liveness Properties ("Good things eventually happen")

- -- *Every transaction request eventually completes*
- □(transaction_requested → ◊transaction_completed)

- -- *Every authentication attempt gets response*
- □(pin_entered → ◊(authenticated ∨ authentication_failed))

- -- *System progress*
- □(session_started → ◊session_ended)

Summery

- Overview of **Set theory**
- Overview of **Logic**
- Overview of **Linear Temporal Logic**
- Application of set theory and logic in **formal specification**

Let's read and explore

– Modeling in Event-B: System and Software Engineering
by Jean-Raymond Abrial